



CAPSTONE PROJECT REPORT

Report 2 – Project Management Plan

A. Record of Changes

A. Record of Changes.....	2
I. Record of Changes.....	3
II. Project Management Plan.....	4
1. Overview.....	4
1.1. Scope & Estimation.....	4
1.2. Project Objectives.....	5
1.2.1. Quality.....	5
1.2.2. Project Objectives.....	6
1.2.3. Team Objectives.....	6
1.3. Project Risks.....	6
2. Management Approach.....	7
2.1. Project Process.....	7
2.1.1. Initial Planning Phase.....	8
2.1.2. Iteration 1: Core Customer & Authentication Features.....	8
2.1.3. Iteration 2: Reception, Workshop & Service Operations.....	8
2.1.4. Iteration 3: Management, Reporting & Billing.....	9
2.1.5. Deployment and Final Review.....	9
2.2. Quality Management.....	9
2.2.1. Document Standards.....	10
2.2.2. Coding Standards.....	10
2.3. Training Plan.....	10
3. Project Deliverables.....	11
4. Responsibility Assignments.....	12
5. Project Communications.....	13
6. Configuration Management.....	13
6.1. Document Management.....	13
6.1.1. Documentation Tools.....	13
6.1.2. Document Storage and Organization.....	13
6.1.3. Document Version Control.....	13
6.1.4. Document Delivery.....	14
6.2. Source Code Management.....	14
6.2.1. Version Control System.....	14
6.2.2. Branching Strategy.....	14
6.2.3. Commit Rules.....	14
6.2.4. Code Review and Merge Process.....	14
6.2.5. Conflict Handling.....	15
6.2.6. Communication Tools.....	15
6.3. Tools & Infrastructures.....	15

I. Record of Changes

Date	A* M, D	In charge	Change Description
14/01/2026	A	KhanhNN	Create new document
21/01/2026	M	KhanhNN	Write document
28/01/2026	M	KhanhNN	Write document

*A - Added M - Modified D - Deleted

II. Project Management Plan

1. Overview

1.1. Scope & Estimation

#	WBS Item	Complexity	Est. Effort (man-days)
1	Authenticate User		20
1.1	Register Account	Medium	5
1.2	Sign In Account	Medium	5
1.3	Sign Out Account	Simple	2
1.4	Change Password	Medium	4
1.5	Forgot Password	Medium	4
2	Manage Personal Service		18
2.1	View Landing Page	Simple	3
2.2	View Service History	Medium	5
2.3	Track Service Progress	Complex	10
3	Manage Personal Bookings	Complex	10
4	Update Personal Profile		10
4.1	View User Profile	Simple	3
4.2	Manage Personal Profile	Medium	7
5	View Attendance Record		16
5.1	Daily Attendance (Check-in / Check-out)	Complex	8
5.2	View My Schedule	Medium	8
6	Process Booking Request		25
6.1	Book Service Appointment	Complex	10
6.2	Create a Walk-in Booking	Complex	15
7	Manage Confirm Booking	Complex	18
7.1	Apply Promotion to Booking	Medium	8
7.2	Queue, reorder, swap Booking	Complex	10
8	Check-in Customer Vehicle	Complex	12
9	Manage Service Ticket		53
9.1	Confirm Payment Status	Medium	8
9.2	Billing by Service Ticket	Complex	12
9.3	Create Service Ticket	Complex	15
9.4	Update Service Ticket	Complex	10
9.5	View Service Ticket	Complex	8
10	Manage Customer Vehicle	Medium	10

11	Manage Customer Profile	Medium	8
12	Manage Customer Queue	Medium	10
13	Notify Customer Reminder		18
13.1	Integrate Zalo OA API	Complex	10
13.2	Send Booking Reminder Message	Simple	4
13.3	Send Service Status Notification	Simple	4
14	Manage Service Quotation		35
14.1	View Service Requests	Simple	5
14.2	Create & Customize Service Estimate	Complex	15
14.3	Create Supplementary Service Quotation	Complex	10
14.4	Stock Allocation	Medium	5
15	Work Assigned Management	Complex	12
16	Manage Work Shifts	Complex	12
17	Manage Staff Profile		18
17.1	View My Work History	Simple	5
17.2	View Assigned Tasks	Medium	7
17.3	Manage Employee Profiles	Simple	6
18	Manage Service Price	Medium	8
19	Manage Customer Promotion	Complex	7
20	Manage Customer Service History		15
20.1	View Customer Service History	Medium	5
20.2	View Dashboard	Complex	10
21	Manage User Accounts	Medium	5
21.1	Manage Roles & Permissions	Complex	10
22	Manage Customer Feedback	Simple	3
23	Manage Personal Vehicles	Medium	4
24	Manage System Report	Complex	5
24.1	View Performance Reports	Simple	2
25	Submit Service Feedback	Simple	2
26	Warehouse Management	Complex	30
26.1	Stock Entry & Issue	Complex	16
26.2	Warehouse Inventory	Medium	6
26.3	Catalog / Item / Brand	Medium	5
26.4	Excel Import/Export	Simple	3

Total Estimated Effort(man-days): 367

1.2. Project Objectives

1.2.1. Quality

#	Testing Stage	Test Coverage	No. of Defects	% of Defect	Notes
1	Requirement Review	80%	≤ 20		Review and validate functional requirements, use case list, and scope definition documents to ensure clarity, completeness, and consistency before development.
2	Unit Test	80%	≤ 20		Unit tests are written and executed by developers to verify that individual modules (e.g., login, booking, service ticket) function correctly according to the design.
3	Integration Test	95%	≤ 100		Test the interaction between integrated modules such as booking management, service tickets, technician assignment, and billing to ensure smooth data flow across the system.
4	System Test	100%	≤ 20		Comprehensive testing of the complete web-based system to verify functionality, behavior, performance, and compliance with the defined functional requirements of the garage management system.
5	User Acceptance Test (UAT)	100%	≤ 5		End users (store owner, receptionist, technician) perform testing based on real operational scenarios such as walk-in booking, vehicle check-in, service execution, and payment confirmation.

1.2.2. Project Objectives

- The project will be completed according to the planned schedule.
- The project will be completed within the allocated effort of 367 man-days.
- The system will fully satisfy the defined functional and non-functional requirements.
- The project will ensure acceptable quality standards through defined testing stages and reviews.

1.2.3. Team Objectives

- The team completed the project before April 30th 2026.
- Each team member follows the assigned roles and tasks according to the project plan.
- Each team member improves both technical skills (system design, implementation, testing) and soft skills (communication, teamwork, time management).
- The team successfully applies a structured software development process throughout the project lifecycle.
- The team collaborates effectively to ensure system quality and project success.

1.3. Project Risks

#	Risk Description	Impact	Possibility	Response Plans
1	Lack of experience with complex business workflows	High	Medium	Analyze business processes carefully at the beginning of the project. Break down complex workflows into smaller tasks. Conduct regular reviews with stakeholders to validate workflow logic.
2	Inaccurate effort estimation for core modules	High	High	Prioritize core features early in the development phase. Continuously track actual effort versus planned effort and adjust the schedule accordingly. Reserve buffer time for critical modules such as Service Ticket and Service Estimation.
3	Integration issues between frontend and backend	High	Medium	Define clear API contracts and data formats early. Perform incremental integration testing after completing each major module. Maintain consistent documentation for APIs and data models.
4	Requirement changes during development	Medium	High	Establish a clear change request process. Evaluate the impact of each change on timeline and effort before approval. Focus on implementing core features first and defer optional features if needed.
5	Data inconsistency between customer, booking, and vehicle records	Medium	Medium	Design a centralized data model with proper relationships and constraints. Apply validation rules at both application and database levels to prevent incorrect data entry.
6	Security vulnerabilities in authentication and role management	High	High	Apply secure authentication mechanisms, password encryption, and role-based access control. Conduct security reviews and testing for authentication, authorization, and sensitive data access.
7	Performance issues when handling multiple concurrent	Medium	Low	Optimize database queries and apply indexing where necessary. Perform performance testing for booking and

	bookings and service updates			service-related functions. Monitor system performance during development.
8	Team members' unfamiliarity with tools or technologies used in the project	Medium	Medium	Allocate time for self-learning and technical discussions. Share knowledge through documentation and internal demos. Assign tasks based on individual strengths and learning progress.
9	Lack of experience with modern web frameworks due to the academic curriculum.	High	High	Conduct a mandatory self-training phase for all members before the official project kickoff. Divide the team into Frontend and Backend roles early so members can focus their learning efficiently.

2. Management Approach

2.1. Project Process

In this project, we apply the Iterative Incremental development model to ensure flexibility, early delivery of core functionalities, and continuous improvement based on feedback. This approach allows the team to gradually build the system while managing risks effectively, especially for complex business workflows such as booking management, service ticket processing, and workshop operations.

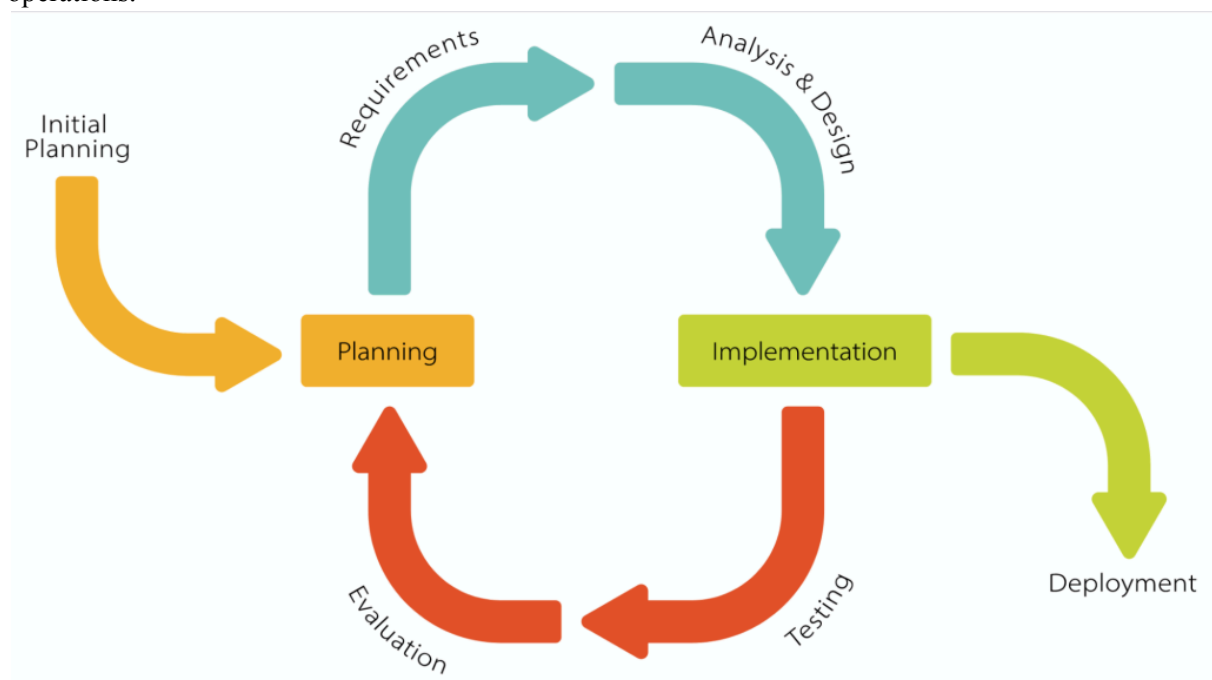


Figure 2-1: Project Process

The project officially starts on January 5th 2026 and is divided into five main iterations, each focusing on specific functional modules of the Garage Management System.

2.1.1. Initial Planning Phase

Duration: [Week 1 – Week 3]

Focus:

- Define project scope, objectives, and system boundaries
- Identify stakeholders and user roles
- Collect and analyze functional and non-functional requirements
- Design high-level system architecture and database schema
- Plan iterations, task allocation, and estimated effort

2.1.2. Iteration 1: Core Customer & Authentication Features

Duration: [Week 4 – Week 5] (26/01/2026 – 08/02/2026)

Focus:

- Develop Authentication & Account Management features
- Implement Customer Services modules such as:
 - + Service catalog viewing
 - + Online service appointment booking
 - + Personal booking management
 - + Vehicle management and service history
- Implement basic UI and navigation flow
- Perform unit testing and integration testing for Iteration 1 features
- Fix bugs and refine functionalities based on internal feedback

2.1.3. Iteration 2: Booking & Reception Operations

Duration: [Week 6 – Week 7] (09/02/2026 – 01/03/2026) *(includes Lunar New Year break)*

Focus:

- Develop Reception & Booking Management modules:
 - + Walk-in booking
 - + Vehicle check-in
 - + Service ticket creation
 - + Technician assignment
- Implement Workshop & Technician Operations
 - + Task assignment
 - + Service status updates
 - + Technical notes management
- Develop Service Advisory & Estimation features
- Conduct integration testing across customer and staff workflows
- Fix defects and optimize system performance

2.1.4. Iteration 3: Service Ticket & Workshop Operations

Duration: [Week 8 – Week 9] (02/03/2026 – 15/03/2026)

Focus:

- Develop Service Ticket Management:
 - + Create, update, and track service tickets
 - + Assign technicians
- Implement Workshop operations:
 - + Task assignment and tracking
 - + Service progress updates
 - + Technician notes
- Develop Service Advisory & basic estimation features
- Ensure end-to-end workflow from check-in service execution
- Conduct integration testing across modules

2.1.5. Iteration 4: Estimation, Warehouse & Advanced Operations

Duration: [Week 10 – Week 11] (16/03/2026 – 29/03/2026)

Focus:

- Implement Service Quotation & Estimation:
 - + Create and update estimates
 - + Supplementary service requests
- Develop Warehouse Management:
 - + Stock entry, stock issue, return handling
 - + Inventory tracking
 - + Stock allocation for service tickets
- Integrate estimation with inventory system

- Optimize system performance for complex workflows
- Conduct integration testing for warehouse and estimation modules

2.1.6. Iteration 5: Management, Billing & System Administration

Duration: [Week 12 – Week 13] (30/03/2026 – 12/04/2026)

Focus:

- Implement Management features:
 - + Staff management and work shifts
 - + Role & permission management
- Develop Pricing & Promotion modules
- Implement Billing & Payment processing
- Build Dashboard & Reporting features
- Conduct system testing and user acceptance testing (UAT)
- Fix major defects and improve usability

2.1.7. Finalization & Review

Duration: Week 14 (13/04/2026 – 20/04/2026)

Focus:

- Perform final system testing and bug fixing
- Review and finalize project documentation
- Clean up source code and optimize performance
- Prepare user manuals and presentation materials

2.1.8. Deployment & Maintenance

Duration: Week 15 (21/04/2026 – 29/04/2026)

Focus:

- Deploy system to production environment
- Monitor system performance and stability
- Fix post-deployment issues
- Provide basic maintenance and support

2.2. Quality Management

Improving project quality and achieving quality objectives require a systematic quality management approach that is integrated throughout the project lifecycle. Since the project follows an iterative incremental methodology, quality assurance activities are continuously applied in each iteration.

Define Clear Quality Objectives

The project aims to deliver a stable, reliable, and user-friendly garage management system that meets functional requirements and supports real-world workshop operations. Quality objectives include correctness, usability, maintainability, and performance.

Requirements Gathering and Analysis

Quality assurance begins at the requirement phase. Business requirements are carefully analyzed to ensure they are complete, consistent, and unambiguous. Clear use case definitions and scope boundaries help prevent misunderstanding and scope creep.

Defect Prevention

Proactive defect prevention measures are applied throughout development, including:

- Reviewing requirement and design documents before implementation
- Applying consistent coding standards and best practices
- Conducting peer reviews for critical features
- Designing modular components to reduce complexity and dependency

Reviewing

Regular reviews are conducted at multiple levels:

- Requirement and design reviews during analysis
- Code reviews during implementation
- Iteration reviews at the end of each development cycle

These reviews help detect issues early and improve overall system quality.

Unit Testing

Developers perform unit testing to verify that individual functions and modules work correctly according to the design. Unit tests focus on authentication logic, booking workflows, service ticket processing, and data validation.

Integration Testing

Integration testing is conducted to ensure that different modules (Customer, Receptionist, Technician, Manager) interact correctly. Special attention is given to workflows such as booking, check-in, service ticket, billing.

System Testing

System testing evaluates the entire application as a complete system. Functional testing, workflow validation, and basic performance checks are conducted to ensure the system meets all specified requirements.

Risk Management

Potential risks related to technical complexity, data consistency, and integration issues are continuously monitored. Mitigation plans are prepared to minimize their impact on system quality.

Documentation and Knowledge Sharing

Project documents are kept up-to-date and shared among team members to avoid knowledge silos. Clear documentation supports maintenance and future system extension.

2.2.1. Document Standards

- All project documents follow the FPT template
- Fonts, font sizes, alignment, and formatting are standardized
- Diagrams and tables are clearly labeled and consistently styled

2.2.2. Coding Standards

- Package, class, method, and variable names are clear and meaningful
- Naming conventions are consistently applied across the project
- Code is structured for readability, maintainability, and reuse
- Comments are added for complex logic and business rules

2.3. Training Plan

Training Area	Participants	When, Duration	Waiver Criteria
Java Spring Boot	Backend Development Team	01/12/2025 - 30 days	Mandatory

ReactJs	Frontend Development Team	01/12/2025 - 30 days	Mandatory
Testing	Tester	01/12/2025 - 30 days	Mandatory

3. Project Deliverables

#	Deliverable	Due Date	Notes
1	Report1_Project Introduction	21/01/2026	ProductBackground ExistingSystems BusinessOpportunity ProductVision ProjectScope&Limitations
2	Report2_Project Management Plan	28/01/2026	ManagementApproach ProjectDeliverables ResponsibilityAssignments ProjectCommunications ConfigurationManagement
3	Report3_Software Requirement Specification	05/02/2026	BusinessRules UseCaseDiagram&UseCase Description FunctionalRequirements Non-FunctionalRequirement
4	Report4_Software Design Document	05/03/2026	Project&SRSReview,Project Review SystemArchitecture Back-endArchitecture Front-endArchitecture
5	Initial Planning Prototype	25/01/2026	
6	Code Iteration 1 Plan and Prototype	08/02/2026	DetailedDesign Code&UnitTest Integration&SystemTestCases
7	Code Iteration 2 Plan and Prototype	01/03/2026	DetailedDesign Code&UnitTest Integration&SystemTestCases
8	Code Iteration 3 Plan and Prototype	15/03/2026	DetailedDesign Code&UnitTest Integration&SystemTestCases
9	Code Iteration 4 Plan and Prototype	29/03/2026	DetailedDesign Code&UnitTest Integration&SystemTestCases
10	Code Iteration 5 Plan and Prototype	12/04/2026	DetailedDesign Code&UnitTest Integration&SystemTestCases
11	Report5_Test Documentation	16/04/2026	TestModel TestPlan TestCases

			TestReports
12	Report6_Software User Guides	17/04/2026	DeliverablePackage InstallationGuidesUser Manual
13	Report7_Final Project Report	25/04/2026	Includesallthefinalprojectdocuments (completedReportsNo.1-6,bothsoft copy&hardcopy) PresentationSlide SourceCode
14	Source Code and Database Script	26/04/2026	

4. Responsibility Assignments

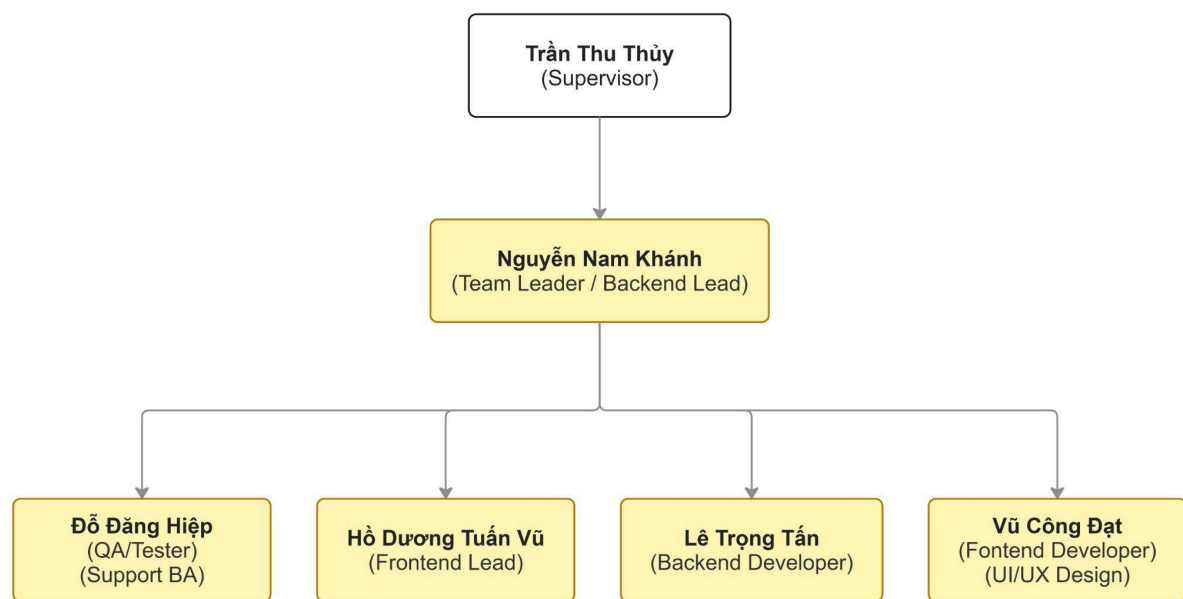


Figure 4-1: Responsibility Assignments

R~Responsible; A~Accountable; C~Consulted; I~Informed;

#	Responsibility	PM	BA	Designer	Frontend dev	Backend dev	Tester	Lecturer	Garage owner
1	Create project chapter	R/A	C	C	C	C	C	C	I
2	Prepare Project Introduction Document	R/A	R/C	I	I	I	I	I/C	C
3	Project Planning and Tracking	R/A	C	I	I	I	I	I/C	I/C
4	Create SRS Document	A	R	C	C	C	I/C	I/C	I/C
5	Create SDD Document	A/C	C	C	R	R	I/C	I/C	I
6	Design Prototype	C	A/C	R	I	I	I	C	I/C

7	Database Analyse	I/C	C	I/C	I/C	R/A	I/C	C	I/C
8	Implement Coding Backend	I/C	C	I	C	R/A	I/C	C	I
9	Implement Coding Frontend	C	C	I	R/A	C	I/C	C	I
10	Create Test Scenario / Test Case	I/C	C/A	I/C	R	R	R	C	I
11	Perform Testing	I/C	I	I	C	C	R/A	I/C	I
12	Create Product Backlog	R	A	I/C	I/C	I/C	I/C	C	I
13	Create Iteration Backlog	R	A	I/C	I/C	I/C	I/C	C	I
14	Create User Manual	A	R	C	C	C	C	C	I/C
15	Prepare Weekly Report and Meeting Report	R/A	R	I	I	I	I	I	I

5. Project Communications

Communication Item	Who/ Target	Purpose	When, Frequency	Type, Tool, Method(s)
Weekly Meeting	All team members and lecturer	Review and determine next week's job	20:00 Every Wednesday	Google Meet
Team daily meeting	All team members	Report project's progress and status of each member's work each day.	21:00 every Monday and Friday, 22:00 Every Tuesday, Thursday	Google Meet
Team daily working	All team members	Work together	Everyday	Messenger, Zalo, Google Sheet
Unscheduled meeting	All team members	Quick meet if serious issues detected	Quick meet if serious issues detected	Google Meet

6. Configuration Management

Configuration Management is applied to control, track, and manage changes of project documents and source code during the software development lifecycle. This helps ensure consistency, traceability, and collaboration among team members.

6.1. Document Management

To effectively manage project documents and their versions, the following document management process is applied:

6.1.1. Documentation Tools

The project uses the following tools to create, edit, and store documents:

- Google Docs, Sheets, Drive
- Miro
- Visual Paradigm

These tools support collaboration and version tracking.

6.1.2. Document Storage and Organization

All project documents are stored centrally on Google Drive.

Documents are organized into separate folders based on document type, such as:

- Project Planning
- SRS (Software Requirements Specification)

- SDD (Software Design Document)
- Test Documents
- User Manual

Access permissions are controlled to ensure only authorized members can edit critical documents.

6.1.3. Document Version Control

Newly created documents start with version 1.0.

Edited documents will be updated as version 1.x

Major changes will increase the version number to 2.0, 3.0, etc.

Change history is recorded directly in the document or via Google Drive version history.

6.1.4. Document Delivery

All final reports are delivered to the supervisor via Google Drive.

Continuous integration is applied for the final report by merging all related documents from planning, analysis, design, implementation, and testing phases.

6.2. Source Code Management

To manage the project source code and its changes effectively, the following source code management process is applied

6.2.1. Version Control System

- Git is used as the version control system.
- The source code repository is hosted on GitLab (or GitHub, depending on team decision).

6.2.2. Branching Strategy

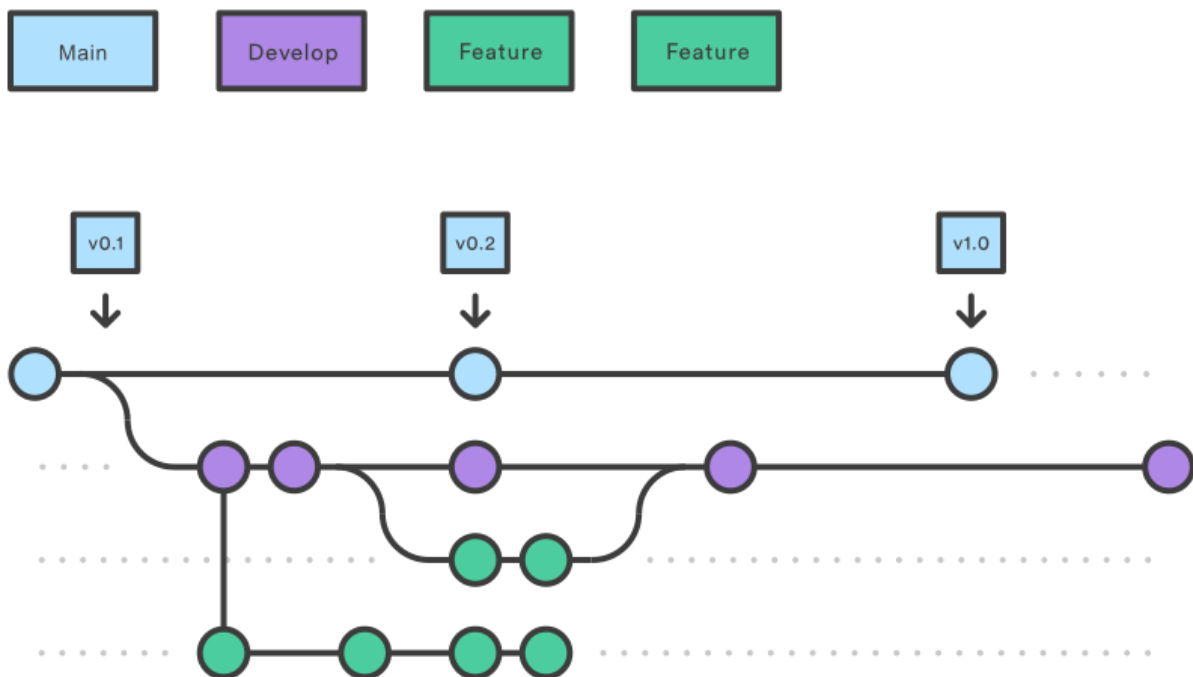


Figure 6-1: GitFlow of MST-GMS Project

The project follows the Gitflow branching model, including:

- Main:

Contains stable and production-ready source code. Direct commits are not allowed.

- Develop:

Contains the latest development code where features are integrated.

- Feature/*:

Each feature or function is developed in a separate branch.

6.2.3. Commit Rules

- Each commit must have a clear and meaningful message describing the change.
- Commits should focus on a single task or function.

- Commit format example:
+ <type>: <short description>

6.2.4. Code Review and Merge Process

- Before merging code from a feature branch into the develop branch, a code review must be performed.
- Pull Requests (Merge Requests) are used to manage merging.
- The team leader or assigned reviewer is responsible for reviewing and approving the code.
- Squash commits are not allowed to ensure traceability.

6.2.5. Conflict Handling

- If conflicts occur during merging, the team leader coordinates with related members to resolve them.
- Conflicts are resolved carefully to maintain code consistency and system stability.

6.2.6. Communication Tools

To support coordination and quick problem-solving during development:

- Zalo / Messenger / Discord: Daily communication
- Google Meet: Online meetings and code review discussions

6.3. Tools & Infrastructures

Category	Tools / Infrastructure
Technology	ReactJs (FrontEnd), Java/SpringBoot (BackEnd)
Database	MySQL
IDEs/Editors	Visual Code, IntelliJ, MySQL Workbench
Diagramming	Visual Paradigm, Miro, ERD plus
Documentation	Google Docs/Sheets/Slides/Drive/Canva
Version Control	GitHub (Source Codes), Google Drive (Documents)
Deployment server	VPS, Docker, Ubuntu
Project management	Google Sheet